TEC | Tecnológico de Costa Rica

Instituto Tecnológico de Costa Rica

Escuela de Computación

Compiladores e Intérpretes - IC5701

Proyecto #2

Análisis Semántico y Generación de Código Intermedio

Estudiantes:

Alice Arias Salazar | 2023104639

Heldyis Agüero Espinoza | 2023296812

Profesor:

Allan Rodriguez Davila

Grupo 60

Entrega: lunes 9 de noviembre 2026

II Semestre 2026

Contenido

1. Requisitos necesarios 3
Paso 1. Verificar los programas necesarios 3
2. Obtener el proyecto desde GitHub 3
Paso 2. Abrir el repositorio en el navegador 3
Paso 3. Descargar el proyecto como ZIP 4
Paso 4. Encontrar el archivo ZIP descargado 4
Paso 5. Extraer el proyecto 5
Paso 6. Identificar la carpeta correcta 5
4. Abrir el proyecto en Visual Studio Code 5
Paso 8. Iniciar Visual Studio Code 5
Paso 9. Seleccionar la carpeta del proyecto 5
5. Verificar que las librerías ya están incluidas 5
Paso 10. Abrir la carpeta Programa 5
Paso 11. Comprobar la carpeta librerías 6
Paso 12. No modificar la carpeta librerias 6
6. Verificar Java 6
Paso 13. Abrir la terminal de Visual Studio Code 6
Paso 14. Comprobar Java 7
7. Conocer la estructura del proyecto 7
Paso 15. Identificar los archivos principales 7
8. Importante: no modificar bin 8
Paso 16. Entender para qué sirven las carpetas que se llenan solas 8
9. Compilar el proyecto 9
Paso 17. Entrar a la carpeta de ejecutables 9
Paso 18. Ejecutar build.bat 9
10. Ejecutar el programa 10
Paso 19. Ejecutar run.bat 10
Paso 20. Seleccionar un archivo .cmm 10

11. Escribir un archivo .cmm 11
Paso 21. Conocer los símbolos del lenguaje 11
Paso 22. Ver un programa de ejemplo 11
Paso 23. Recordar las reglas más importantes 12
12. Entender lo que muestra el programa 12
Paso 24. Análisis léxico 12
Paso 25. Generación del archivo de tokens 12
Paso 26. Análisis sintáctico 13
Paso 27. Análisis semántico 13
Paso 28. Código intermedio 14
Paso 29. Resultado final 14
Paso 30. Responder las preguntas finales 15
Paso 31. Saber qué archivos crea el programa 16
13. ¿Qué hacer si aparece un error? 16
Paso 32. Error durante la compilación 16
Paso 33. Error relacionado con Java 16
Paso 34. Error en un archivo .cmm 16
Paso 35. El programa dice que no generó código intermedio 16
Paso 36. Falta la función principal 17
Pruebas de funcionalidad 17
Descripción del problema 26
Diseño del programa 28
Librerías usadas 31
Análisis de resultados 32
Justificación de toma de decisiones 33
Bitácora 35

Manual de usuario

El proyecto se encuentra guardado en el repositorio de GitHub del equipo:

https://github.com/Alice-Arias/PY2-Analisis-Semantico-y-Generacion-Codigo-Intermedio-Compiladores

El programa recibe un archivo de código con extensión .cmm y realiza cuatro fases. Primero, separa el texto en palabras, números y símbolos (análisis léxico). Después, revisa que estén ordenados correctamente (análisis sintáctico). Luego, comprueba que el programa tenga sentido y que los tipos de datos se utilicen correctamente (análisis semántico). Finalmente, si no encuentra errores, genera un archivo con una versión más sencilla del programa, llamada código intermedio.

El proyecto está desarrollado en Java y utiliza JFlex para el análisis léxico y Java CUP para los análisis sintáctico y semántico. Ambas librerías ya están incluidas en el repositorio, por lo que no es necesario descargarlas por separado.

1. Requisitos necesarios

Paso 1. Verificar los programas necesarios

Para usar el proyecto se necesita una computadora con Windows, Visual Studio Code y Java JDK 21. También se puede usar Git si se quiere copiar el proyecto con comandos, pero no es obligatorio: el proyecto también se puede descargar como un archivo ZIP. No hace falta instalar JFlex ni Java CUP. Las versiones que usa el proyecto ya están en la carpeta Programa\librerias.

2. Obtener el proyecto desde GitHub

Paso 2. Abrir el repositorio en el navegador

Primero se abre un navegador, como Google Chrome, Microsoft Edge o Mozilla Firefox. En la barra de direcciones se escribe:

```
https://github.com/Alice-Arias/PY2-Analisis-Semantico-y-Generacion-Codigo-Intermedio-Compiladores
```

y se presiona Enter. Se abrirá la página del repositorio, donde se ven las carpetas y los archivos del proyecto.

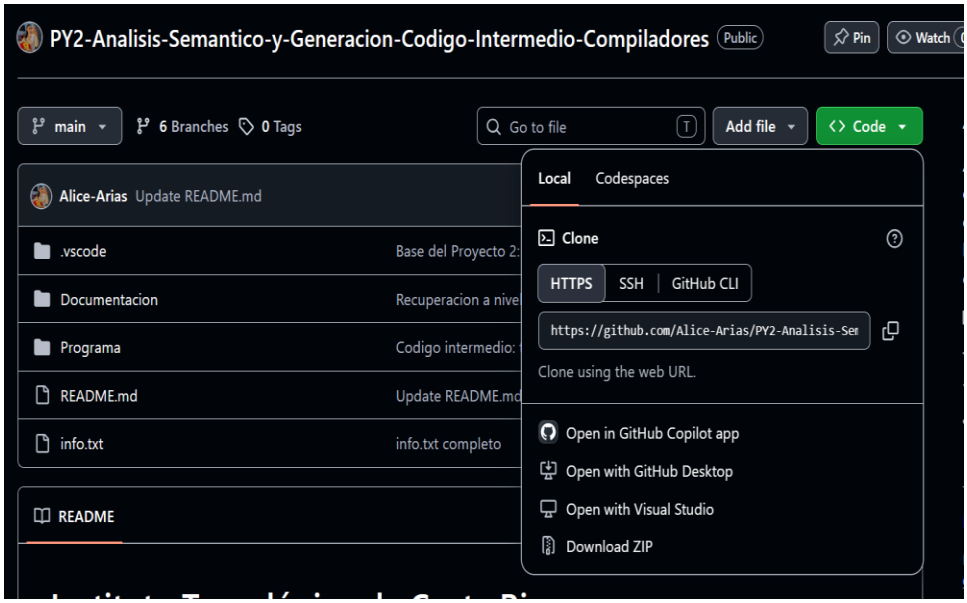
Paso 3. Descargar el proyecto como ZIP

Para una persona que no conoce Git, esta es la forma más fácil. En la página del repositorio se busca el botón verde que dice Code. Al presionarlo aparece un menú. Dentro de ese menú se elige:

`Download ZIP`

El navegador descargará un archivo comprimido con una copia completa del proyecto. Su nombre es parecido a:

`PY2-Analisis-Semantico-y-Generacion-Codigo-Intermedio-Compiladores-main.zip`



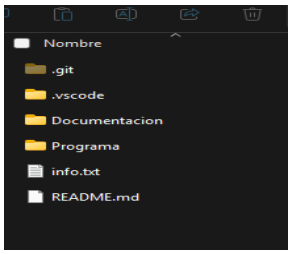
Paso 4. Encontrar el archivo ZIP descargado

Normalmente el archivo se guarda en la carpeta Descargas de Windows. Se abre el explorador de archivos, se entra en Descargas y se busca el archivo ZIP. Todavía está comprimido, así que no se debe intentar abrir el proyecto desde dentro del ZIP.

Paso 5. Extraer el proyecto

Se hace clic derecho sobre el archivo ZIP y se elige Extraer todo…. Windows pregunta dónde se quiere guardar el proyecto. Para hacerlo más fácil se puede elegir el Escritorio. Después se presiona Extraer. Windows creará una carpeta con todo el proyecto.

Paso 6. Identificar la carpeta correcta



Después de extraer el ZIP hay que entrar a la carpeta que realmente contiene el proyecto. Al abrirla se deben ver elementos como estos:

Si al abrir la carpeta solo aparece otra carpeta con el mismo nombre, se entra en ella hasta encontrar Programa, Documentacion y README.md. Esto es importante porque Visual Studio Code debe abrir la carpeta principal del proyecto.

4. Abrir el proyecto en Visual Studio Code

Paso 8. Iniciar Visual Studio Code

Se abre Visual Studio Code. No se debe crear un proyecto Java nuevo, porque el proyecto ya está creado y trae todos sus archivos. Se elige:

`File → Open Folder…`

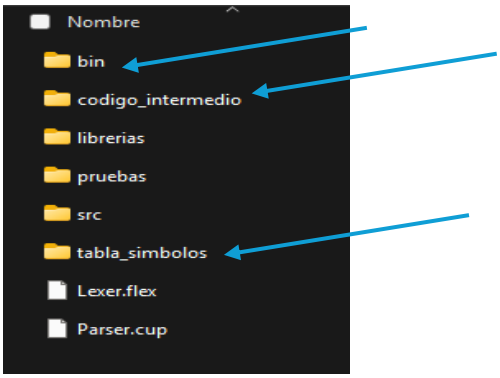
Paso 9. Seleccionar la carpeta del proyecto

En la ventana que aparece se busca la carpeta del proyecto y se presiona Select Folder. Visual Studio Code cargará los archivos y los mostrará en el explorador, a la izquierda de la pantalla.

5. Verificar que las librerías ya están incluidas

Paso 10. Abrir la carpeta Programa

En el explorador de Visual Studio Code se abre la carpeta Programa. Dentro se deben encontrar estas carpetas y archivos:



Las carpetas bin, codigo_intermedio y tabla_simbolos se llenan solas cuando se usa el programa. Si alguna no aparece al principio, se crea sola la primera vez que se necesita.

Paso 11. Comprobar la carpeta librerías

Dentro de Programa se abre la carpeta librerias. Ahí están guardadas las herramientas JFlex 1.9.1 y Java CUP 11b, así que no hay que entrar a Internet a descargarlas.



Paso 12. No modificar la carpeta librerias

Aunque las librerías ya estén incluidas, no se deben cambiar. No se les debe cambiar el nombre, ni moverlas a otra carpeta, ni reemplazarlas por versiones descargadas de Internet. Los archivos build.bat y run.bat las buscan en Programa\librerias y, si se mueven, dejarían de encontrarlas.

6. Verificar Java

Paso 13. Abrir la terminal de Visual Studio Code

Aunque JFlex y Java CUP ya vengan con el proyecto, Java sí debe estar instalado en la computadora. En Visual Studio Code se elige:

```
Terminal → New Terminal
```

En la parte de abajo aparecerá una terminal, que es una ventana donde se escriben comandos.

Paso 14. Comprobar Java

En la terminal se escribe este comando y se presiona Enter:

```
java -version
```

Después se escribe este otro y se presiona Enter:

```
javac -version
```

El segundo comando es muy importante porque comprueba que el compilador de Java esté disponible. Un resultado correcto se parece a:

```
java version "21.0.12"
```

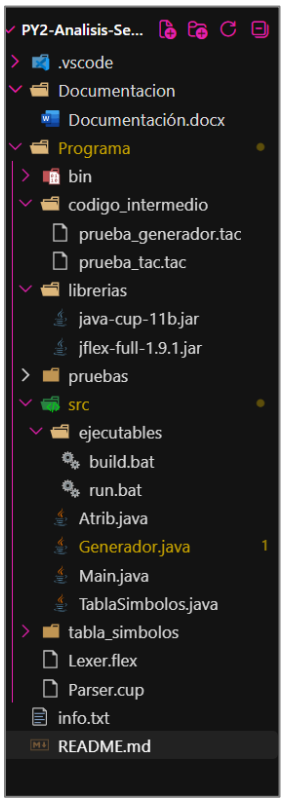
```
javac 21.0.12
```

El número exacto puede cambiar si se usa otra actualización de Java 21. Si los dos comandos funcionan, la computadora está lista.

7. Conocer la estructura del proyecto

Paso 15. Identificar los archivos principales

La carpeta Programa tiene los archivos más importantes del proyecto. Así se ven en Visual Studio Code:



Estos son los archivos principales:

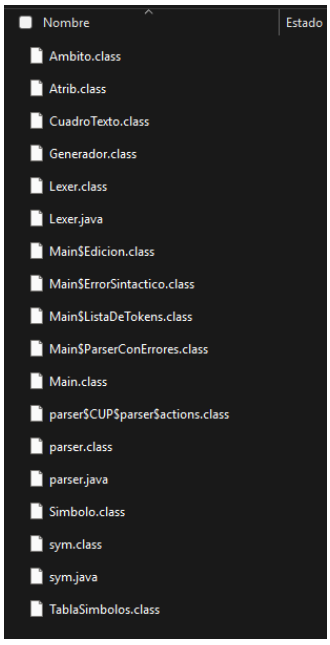
- **Lexer.flex:** tiene las reglas para separar el código en sus partes (palabras, números, símbolos). JFlex lo usa para crear el analizador léxico.
- **Parser.cup:** tiene las reglas del lenguaje y las revisiones de sentido (tipos, nombres, funciones). Java CUP lo usa para crear el analizador sintáctico y semántico. También le pide al generador que escriba el código intermedio.

- **src\Main.java:** es el programa principal. Coordina las cuatro fases y muestra los resultados en pantalla.
- **src\TablaSimbolos.java:** guarda la lista de variables y funciones que aparecen en el programa.
- **src\Atrib.java:** guarda los datos de cada valor mientras se analiza: su tipo y dónde queda su resultado.
- **src\Generador.java:** escribe el código intermedio.
- **src\ejecutables:** tiene los archivos build.bat y run.bat, que sirven para compilar y ejecutar con un solo comando.
- **pruebas:** tiene los archivos .cmm que se usan para probar el programa.
- **librerias:** tiene JFlex y Java CUP.
- **bin:** tiene los archivos que se generan al compilar.
- **tabla_simbolos:** guarda los reportes de la tabla de símbolos.
- **codigo_intermedio:** guarda los archivos con el código intermedio.

8. Importante: no modificar bin

Paso 16. Entender para qué sirven las carpetas que se llenan solas

La carpeta bin se usa cuando se compila el proyecto. Los archivos que hay adentro los crea el propio proceso de compilación, por ejemplo Lexer.java, parser.java, sym.java y varios archivos .class. No se deben editar, mover, cambiar de nombre ni borrar, y tampoco se deben crear archivos nuevos dentro de bin.



Si se cambia Lexer.flex o Parser.cup, los archivos de bin se vuelven a crear solos al compilar de nuevo.

Las carpetas tabla_simbolos y codigo_intermedio son diferentes: ahí el programa guarda los resultados de cada ejecución. Esos archivos sí se pueden abrir y leer, pero no hace falta editarlos.

9. Compilar el proyecto

Paso 17. Entrar a la carpeta de ejecutables

Los archivos que compilan y ejecutan el proyecto no están directamente en Programa. Están en:

```
Programa\src\ejecutables
```

En la terminal se entra primero a Programa:

```
cd .\Programa
```

Después a src:

```
cd .\src
```

Y por último a ejecutables:

```
cd .\ejecutables
```

La terminal debe terminar mostrando una ruta parecida a:

```
PS C:\Users\NombreUsuario\Desktop\PY2-Analisis-Semantico-y-
Generacion-Codigo-Intermedio-Compiladores-
main\Programa\src\ejecutables>
```

Esa es la ubicación correcta para usar los archivos .bat.

Paso 18. Ejecutar build.bat

Desde la carpeta ejecutables se escribe:

```
.\build.bat
```

y se presiona Enter. Este archivo hace todo el trabajo de compilar. Primero busca las librerías que están en Programa\librerias. Después usa Java CUP para crear el analizador del lenguaje, usa JFlex para crear el separador de palabras y, por último, usa javac para compilar los archivos de Java.

Al terminar debe aparecer el mensaje COMPILACION COMPLETADA. También se verá una advertencia que dice que el terminal PRINT fue declarado,

pero no se usa. Es normal: la instrucción print del primer proyecto se cambió por write y su palabra quedó definida sin usarse.

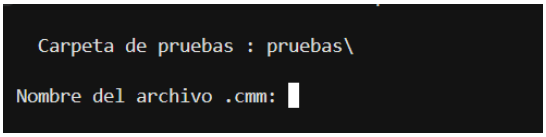
10. Ejecutar el programa

Paso 19. Ejecutar run.bat

Cuando la compilación terminó bien, se sigue en la misma carpeta (Programa\src\ejecutables) y se escribe:

```
.\run.bat
```

y se presiona Enter. Este archivo inicia el programa y pide el nombre del archivo .cmm que se quiere analizar.



Paso 20. Seleccionar un archivo .cmm

Los archivos de prueba están en la carpeta:

```
Programa\pruebas
```

Todos tienen la extensión .cmm. Por ejemplo:

```
prueba_valida_p2.cmm

prueba_semantica.cmm

prueba_recuperacion.cmm

prueba_generador.cmm
```

Cuando run.bat pide el nombre del archivo, se escribe el nombre del que se quiere probar.

También se puede ejecutar el programa directamente, sin run.bat. Desde la carpeta Programa se escribe:

```
java -cp "bin;librerias\java-cup-11b.jar" Main
pruebas\prueba_generador.cmm pruebas\tokens.txt
```

El primer nombre es el archivo que se analiza y el segundo es el archivo donde se guardan las palabras reconocidas.

11. Escribir un archivo .cmm

Paso 21. Conocer los símbolos del lenguaje

El lenguaje usa algunos símbolos especiales en lugar de los que se usan en otros lenguajes, como las llaves o el punto y coma. Estos son los más importantes:

Símbolo	Para qué sirve
¿: y :?	Abren y cierran un bloque (donde en otros lenguajes irían llaves)
ɛ: y :ɜ	Abren y cierran un paréntesis
ʃ: y :ɻ	Abren y cierran un corchete (para el número de posición de un arreglo)
»	Marca el final de una instrucción (como el punto y coma)
⊢	Asignación: guarda un valor en una variable
ʌ θ Σ	Y lógico, O lógico y negación
|	Comienza un comentario de una línea

Paso 22. Ver un programa de ejemplo

Un programa tiene una o más funciones y exactamente una función llamada principal. Este es un ejemplo corto:

```
int doble ɛ: int n :ɜ ¿:
    return n + n»
:?
void principal ɛ: :ɜ ¿:
    val int i ⊢ 0»
    while ɛ: ʃ:i:ɻ < ʃ:3:ɻ :ɜ ¿:
        i++
    :?
    write ɛ: doble ɛ: i :ɜ :ɜ»
:?
```

La función doble recibe un número y devuelve su doble. La función principal crea la variable i con el valor 0, la aumenta de uno en uno hasta llegar a 3 y escribe el doble del resultado.

Paso 23. Recordar las reglas más importantes

- Las variables se crean con la palabra val y siempre llevan su tipo: int, float, bool, char o string.
- Los tipos no se mezclan: un valor decimal no puede guardarse en una variable entera.
- Una función debe escribirse antes de usarla.
- Las condiciones de if, elif, while y for deben ser de tipo bool, es decir, verdadero o falso.
- Los arreglos son de tipo int o float, y la primera posición es la número 0.

12. Entender lo que muestra el programa

Paso 24. Análisis léxico

Después de seleccionar el archivo .cmm, el programa comienza a leerlo e identificar sus elementos, como palabras reservadas, nombres, números, textos, operadores y símbolos. Cada elemento reconocido se llama token. Si encuentra algo que no pertenece al lenguaje, como el carácter @, muestra un error indicando la línea y la columna, y continúa con el análisis.

Paso 25. Generación del archivo de tokens

Durante el análisis se crea el archivo:

`Programa\pruebas\tokens.txt`

Ahí se guardan todas las partes reconocidas, con su línea, su columna y lo que decía el texto. Se puede abrir desde Visual Studio Code después de ejecutar el programa. Este archivo es un resultado de la ejecución y no se debe confundir con los archivos de la carpeta bin.

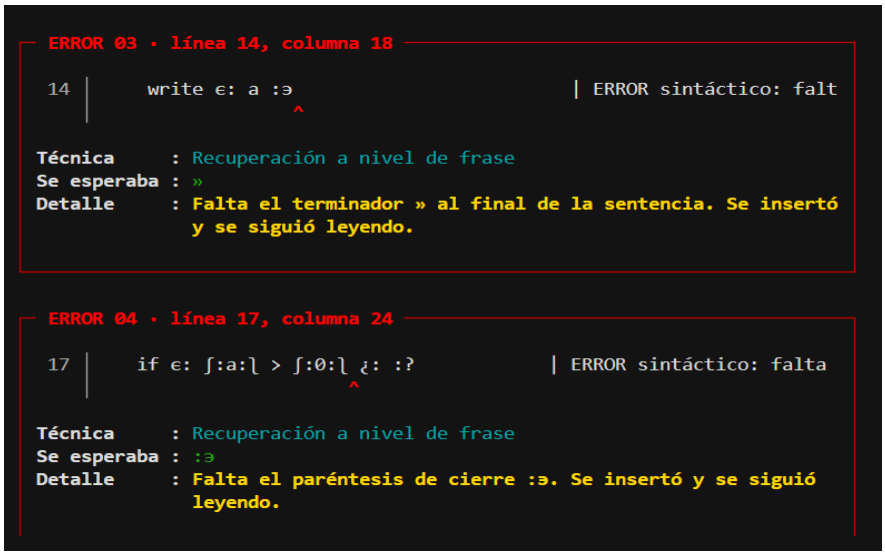
Paso 26. Análisis sintáctico

Con las partes reconocidas, el programa revisa que estén en el orden correcto, según las reglas del lenguaje. Si falta algo, por ejemplo el símbolo » al final de una instrucción, se muestra un recuadro con la línea del problema, una marca ^ en el lugar exacto, lo que se encontró y lo que se esperaba. Cuando el programa logra arreglar el error por su cuenta, también indica cómo lo hizo.

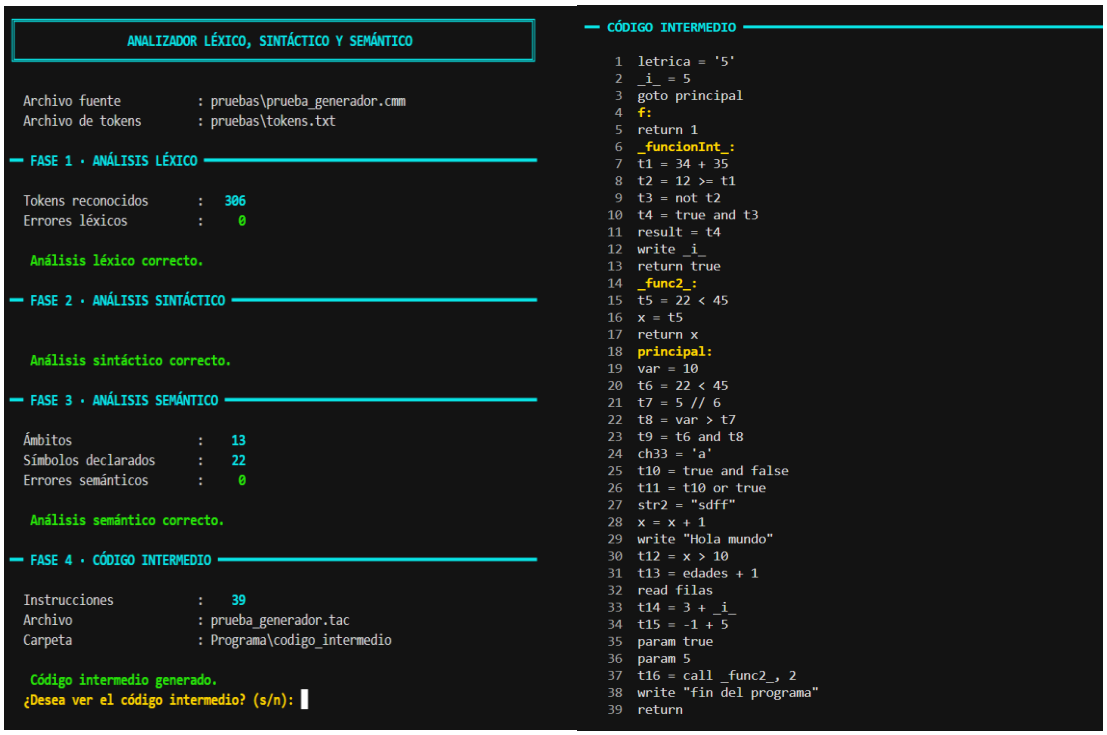
Paso 27. Análisis semántico

Si el orden es correcto, el programa revisa que todo tenga sentido. Por ejemplo, que una variable esté creada antes de usarla, que no se sume un texto con un número, que una función reciba la cantidad de datos que necesita y que la condición de un if sea verdadera o falsa.

Los errores de esta fase se muestran uno por línea, con su número, su línea, su columna, el tipo de revisión que falló (por ejemplo Nombres, Aritmética o Arreglos) y una explicación. Al final aparece un resumen de cuántos errores hubo de cada tipo.



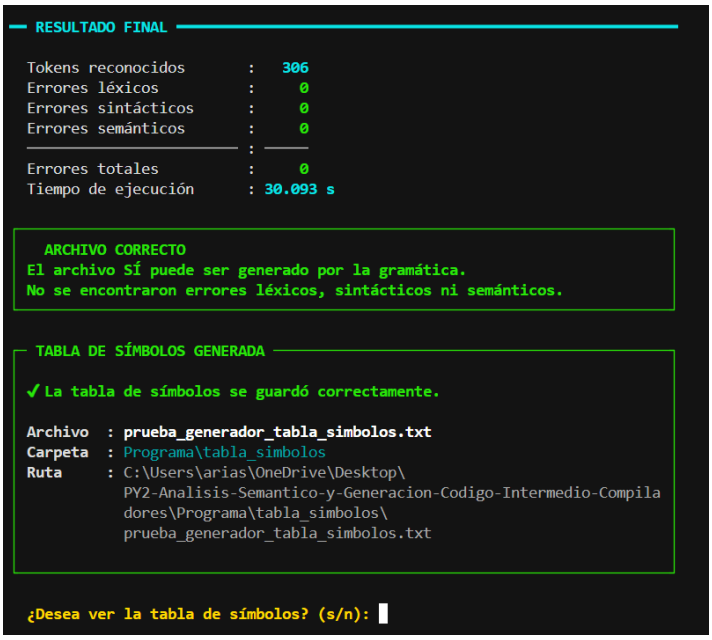
Paso 28. Código intermedio



Si no se encontró ningún error, el programa escribe el código intermedio. Es una versión muy sencilla del programa, en la que cada línea hace una sola operación, por ejemplo t1 = a + b. Se guarda en la carpeta codigo_intermedio con el mismo nombre del archivo analizado y la extensión .tac. Si hay errores, no se escribe ningún código intermedio.

Paso 29. Resultado final

Al final se muestra un resumen con la cantidad de errores de cada fase y el tiempo que tardó el programa. También se indica si el archivo SÍ puede o NO puede ser generado por la gramática, es decir, si el programa está bien escrito y tiene sentido.



Paso 30. Responder las preguntas finales

Al terminar, el programa hace dos preguntas. Se responde s para decir que sí o n para decir que no:

- **¿Desea ver el código intermedio?** Si se responde s, se muestra el código en la pantalla, con cada línea numerada.
- **¿Desea ver la tabla de símbolos?** Si se responde s, se muestra la lista de variables y funciones con todos sus datos.

La tabla de símbolos siempre se guarda también en un archivo, aunque se responda n, si se respondiera s se vería así.

```
¿Desea ver la tabla de símbolos? (s/n): s

── TABLA DE SÍMBOLOS ──────────────────────────────────────────────

Árbol de ámbitos (cada ámbito es una tabla; los de abajo son sus hijos)

global  (8 símbolos)
├── funcion f  (0 símbolos)
├── funcion _funcionInt_  (1 símbolo)
│   └── while  (0 símbolos)
├── funcion _func2_  (3 símbolos)
│   └── if  (1 símbolo)
└── procedimiento principal  (4 símbolos)
    ├── if  (2 símbolos)
    ├── elif  (1 símbolo)
    ├── else  (2 símbolos)
    └── for  (0 símbolos)
        └── cuerpo for  (0 símbolos)
            └── if  (0 símbolos)

Categorías: variable  arreglo  parámetro  función  procedimiento
Dirección: global+n = zona estática · pila+n = registro de activación
Acceso:    público = declarado en el ámbito global · privado = dentro de una función o bloque
```

Nº	NOMBRE	CATEGORÍA	TIPO	ÁMBITO	BYTES	DIMENSIÓN	DIRECCIÓN	VALOR INICIAL	ACCESO	LÍNEA	COL
1	letrica	variable	char	global	1	–	global+0	'5'	público	1	10
2	_arr_	arreglo	int	global	80	[0..19]	global+1	–	público	2	9
3	_i_	variable	int	global	4	–	global+81	5	público	3	9
4	arr	arreglo	int	global	12	[0..2]	global+85	¿: 22, 10, 11 :?	público	4	9
5	f	función	int	global	–	–	código	–	público	8	5
6	_funcionInt_	función	bool	global	–	–	código	–	público	10	6
7	_func2_	función	bool	global	–	–	código	–	público	20	6
8	principal	procedimiento	void	global	–	–	código	–	público	28	6
9	result	variable	bool	función _funcionInt_	1	–	pila+0	e: true :ɜ λ Σ e: e: ∫: 12 :⌊ >= ∫: 34 + 35 :⌊ :ɜ :ɜ	privado	11	14
10	_b1_	parámetro	bool	función _func2_	1	–	pila+0	–	privado	20	23
11	_i1_	parámetro	int	función _func2_	4	–	pila+1	–	privado	20	33
12	x	variable	bool	función _func2_	1	–	pila+5	∫: 22 :⌊ < ∫: 45 :⌊	privado	21	14
13	y	variable	int	función _func2_ › if	4	–	pila+6	–	privado	23	17
14	var	variable	int	procedimiento principal	4	–	pila+0	10	privado	29	13
15	x	variable	int	procedimiento principal	4	–	pila+21	–	privado	41	13
16	edades	arreglo	int	procedimiento principal	12	[0..2]	pila+25	¿: 22, 10, 11 :?	privado	42	13
17	filas	arreglo	int	procedimiento principal	20	[0..4]	pila+37	–	privado	52	13
18	y	variable	int	procedimiento principal › if	4	–	pila+4	–	privado	31	11
19	ch33	variable	char	procedimiento principal › if	1	–	pila+8	'a'	privado	32	12
20	y	variable	int	procedimiento principal › elif	4	–	pila+9	–	privado	35	11
21	y	variable	int	procedimiento principal › else	4	–	pila+13	–	privado	38	11
22	str2	variable	string	procedimiento principal › else	4	–	pila+17	"sdff"	privado	39	14

Paso 31. Saber qué archivos crea el programa

Archivo	Dónde queda	Qué contiene
tokens.txt	Programa\pruebas	Las partes reconocidas del programa
<nombre>_tabla_simbolos.txt	Programa\tabla_simbolos	La lista de variables y funciones, con su tipo, su ámbito y su dirección
<nombre>.tac	Programa\codigo_intermedio	El código intermedio (solo si no hubo errores)

13. ¿Qué hacer si aparece un error?

Paso 32. Error durante la compilación

Si build.bat muestra un error, se debe comprobar que se esté ejecutando desde la carpeta correcta y que existan todas las carpetas y archivos necesarios. No es necesario descargar otras versiones de JFlex o Java CUP, ya que las necesarias están incluidas en el repositorio.

Paso 33. Error relacionado con Java

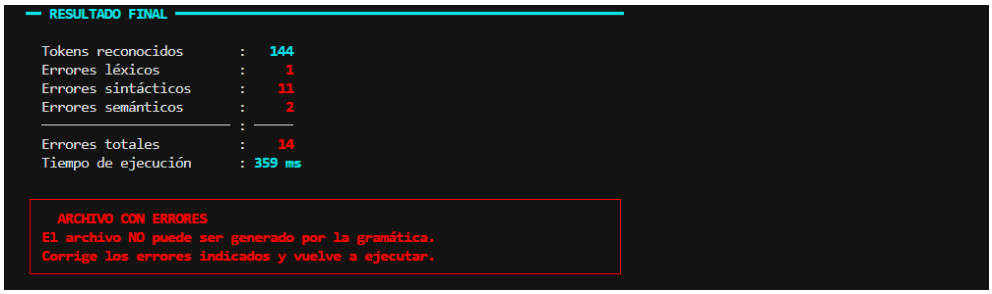
Si aparece un error indicando que java o javac no se reconoce, se debe revisar la instalación de Java y comprobar que ambos comandos funcionen. Si se agrega un archivo .java nuevo, también debe incluirse en la lista de archivos de build.bat para que pueda compilarse correctamente.

Paso 34. Error en un archivo .cmm

Si el programa muestra un error con su línea y columna, se debe buscar esa ubicación en el archivo .cmm y corregir lo que no cumple las reglas del lenguaje. La explicación del mensaje indica cuál es el problema.

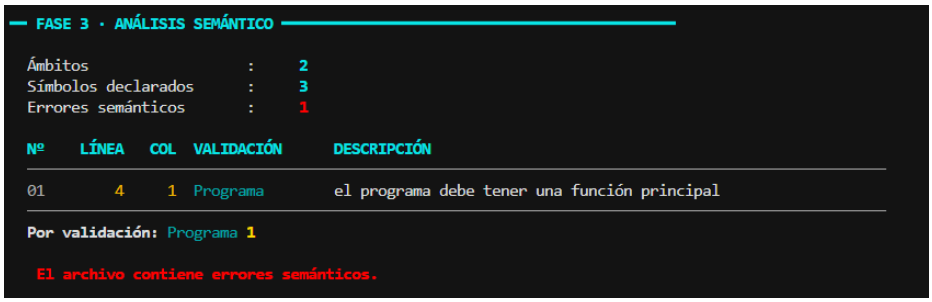
Paso 35. El programa dice que no generó código intermedio

El mensaje No se generó código intermedio: el archivo tiene errores significa que el programa encontró al menos un error en alguna de las fases. Se deben corregir todos los errores que se mostraron y ejecutar el programa otra vez. Cuando el archivo no tenga errores, el código intermedio se escribirá solo.



Paso 36. Falta la función principal

Todo programa debe tener exactamente una función llamada principal. Si no existe, el programa muestra el error el programa debe tener una función principal. Si se escribieron dos, el segundo aparece como un nombre repetido. En ambos casos se corrige el archivo .cmm.



Pruebas de funcionalidad

En esta sección se presentan capturas de pantalla de las pruebas realizadas con diferentes archivos. Cada prueba incluye una breve explicación de lo que se quiere comprobar y de los resultados obtenidos. Todos los archivos .cmm se encuentran en la carpeta Programa\pruebas..

Prueba 1. Compilación del proyecto

Se ejecuta build.bat para comprobar que el proyecto compile correctamente. En la imagen se observa que Java CUP y JFlex generaron los analizadores sin errores y que al final aparece el mensaje COMPILACION COMPLETADA. La advertencia sobre el terminal PRINT es normal, ya que esta instrucción ya no se utiliza en la gramática.

```
PS C:\Users\arias\...\Programa> .\src\ejecutables\build.bat

============================================================
                COMPILACION DEL PROYECTO
============================================================

Proyecto : C:\Users\arias\...\Programa\src\ejecutables\..\..
JFlex    : ...\Programa\librerias\jflex-full-1.9.1.jar
CUP      : ...\Programa\librerias\java-cup-11b.jar
Bin      : ...\Programa\bin

Limpiando archivos generados anteriores...
Limpieza de bin completada.

Generando Parser con CUP...
Warning : Terminal "PRINT" was declared but never used
------- CUP v0.11b 20150326 (SVN rev 63) Parser Generation Summary -------
  0 errors and 1 warning
  54 terminals, 106 non-terminals, and 238 productions declared,
  producing 443 unique parse states.
  1 terminal declared but not used.
  0 non-terminal declared but not used.
  0 productions never reduced.
  0 conflicts detected (0 expected).
  Code written to "parser.java", and "sym.java".
---------------------------------------------------- (CUP v0.11b 20150326 (SVN rev 63))
CUP generado correctamente.

Generando Lexer con JFlex...
Reading "Lexer.flex"
Constructing NFA : 399 states in NFA
Converting NFA to DFA :
166 states before minimization, 147 states in minimized DFA
JFlex generado correctamente.

Compilando archivos Java...

============================================================
             COMPILACION COMPLETADA
============================================================
```

Prueba 2. Un programa correcto

Se ejecuta el archivo prueba_valida_p2.cmm, que contiene un programa grande y sin errores. En la imagen se observa que todas las fases finalizaron correctamente: se reconocieron 1055 tokens, se crearon 25 ámbitos con 85 símbolos y se generaron 305 instrucciones de código intermedio. Al final, todos los errores están en cero y el programa indica que el archivo sí puede ser generado por la gramática.

```
            ANALIZADOR LÉXICO, SINTÁCTICO Y SEMÁNTICO

  Archivo fuente          : pruebas\prueba_valida_p2.cmm
  Archivo de tokens       : pruebas\tokens.txt

── FASE 1 · ANÁLISIS LÉXICO ──────────────────────────────

  Tokens reconocidos      :  1055
  Errores léxicos         :     0

   Análisis léxico correcto.

── FASE 2 · ANÁLISIS SINTÁCTICO ──────────────────────────

   Análisis sintáctico correcto.

── FASE 3 · ANÁLISIS SEMÁNTICO ───────────────────────────

  Ámbitos                 :    25
  Símbolos declarados     :    85
  Errores semánticos      :     0

   Análisis semántico correcto.

── FASE 4 · CÓDIGO INTERMEDIO ────────────────────────────

  Instrucciones           :   305
  Archivo                 : prueba_valida_p2.tac
  Carpeta                 : Programa/codigo_intermedio

   Código intermedio generado.
  ¿Desea ver el código intermedio? (s/n):


── RESULTADO FINAL ───────────────────────────────────────

  Tokens reconocidos      :  1055
  Errores léxicos         :     0
  Errores sintácticos     :     0
  Errores semánticos      :     0
  ──────────────────────  :  ────
  Errores totales         :     0
  Tiempo de ejecución     : 612 ms

    ARCHIVO CORRECTO
  El archivo SÍ puede ser generado por la gramática.
  No se encontraron errores léxicos, sintácticos ni semánticos.
```

Prueba 3. Código intermedio de un programa corto

Se analiza el siguiente programa, que declara variables, hace operaciones y usa read y write:

```
void principal ɛ: :ɜ ¿:

    val int a ⊦ 1»

    val int b ⊦ 2»

    val int c ⊦ 0»

    c ⊦ a + b * 2»

    c ⊦ a mod b»

    read ɛ: a :ɜ»

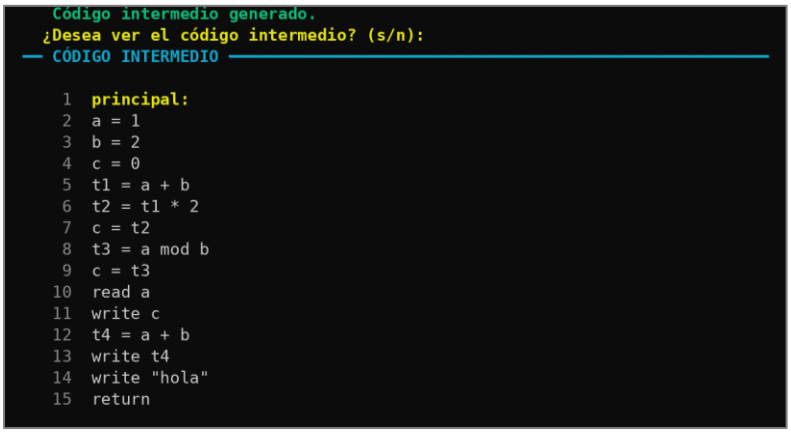
    write ɛ: c :ɜ»

    write ɛ: a + b :ɜ»

    write ɛ: "hola" :ɜ»

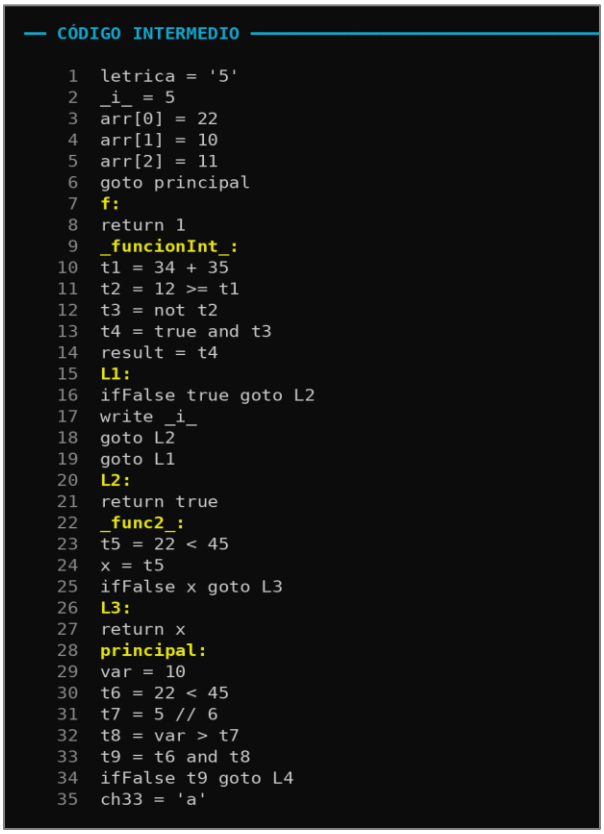
:?
```

En la imagen se muestra el código intermedio generado. Las variables con valores iniciales se convierten en asignaciones. Las operaciones se dividen en pasos más pequeños y utilizan variables temporales (t1, t2, t3, etc.) para guardar resultados parciales. Las instrucciones read y write se mantienen en el código generado. También se muestra la función principal y su instrucción return, que indica el final del programa..



Prueba 4. Código intermedio de un programa completo

Se ejecuta el archivo prueba_generador.cmm, que contiene variables globales, un arreglo, varias funciones, ciclos, condiciones y llamadas a funciones. El programa genera 69 instrucciones de código intermedio. En la imagen se observa que las variables globales y los valores del arreglo aparecen al inicio, seguido de goto principal, que indica dónde comienza la ejecución. Cada función se identifica con su nombre y dos puntos. Las comparaciones se guardan en variables temporales y los ciclos while se representan mediante etiquetas y saltos. La instrucción break se convierte en un salto que permite salir del ciclo.:



En la segunda imagen se ve el resto: el final de los if con elif y else, el for, el acceso a un arreglo (t13 = edades[2]) y la llamada a la función _func2_, que se escribe con un param por cada dato que recibe y un call que guarda el resultado en un temporal.

```
36  goto L5
37  L4:
38  t10 = true and false
39  t11 = t10 or true
40  ifFalse t11 goto L6
41  goto L5
42  L6:
43  str2 = "sdff"
44  L5:
45  edades[0] = 22
46  edades[1] = 10
47  edades[2] = 11
48  x = 0
49  L7:
50  write "Hola mundo"
51  t12 = x > 10
52  ifFalse t12 goto L9
53  t13 = edades[2]
54  t14 = t13 + 1
55  edades[2] = t14
56  goto L8
57  L9:
58  x = x + 1
59  goto L7
60  L8:
61  read filas[0]
62  t15 = 3 + _i_
63  t16 = -1 + 5
64  _arr_[t15] = t16
65  param true
66  param 5
67  t17 = call _func2_, 2
68  write "fin del programa"
69  return
```

Prueba 5. Tabla de símbolos

Al terminar el análisis, el programa pregunta si se quiere ver la tabla de símbolos. En la imagen se ve primero el árbol de ámbitos: el ámbito global y, dentro, el de la función principal. Debajo está la tabla con una fila por cada variable o función: su nombre, su categoría, su tipo, el ámbito donde se creó, los bytes que ocupa ,su desplazamiento en memoria (pila+0, pila+4, etc.)su valor inicial, si es público o privado y la línea y la columna donde se declaró.

— TABLA DE SÍMBOLOS —

Árbol de ámbitos (cada ámbito es una tabla; los de abajo son sus hijos)

global (1 símbolo)
└─ procedimiento principal (3 símbolos)

Categorías: variable arreglo parámetro función procedimiento
Dirección: global+n = zona estática · pila+n = registro de activación
Acceso: público = declarado en el ámbito global · privado = dentro de una función o bloque

Nº	NOMBRE	CATEGORÍA	TIPO	ÁMBITO	BYTES	DIMENSIÓN	DIRECCIÓN	VALOR INICIAL	ACCESO	LÍNEA	COL
1	principal	procedimiento	void	global	–	–	código	–	público	1	6
2	a	variable	int	procedimiento principal	4	–	pila+0	1	privado	2	13
3	b	variable	int	procedimiento principal	4	–	pila+4	2	privado	3	13
4	c	variable	int	procedimiento principal	4	–	pila+8	0	privado	4	13

Prueba 6. Errores léxicos y errores que el programa arregla por frase

Se ejecuta el archivo prueba_recuperacion.cmm, que contiene errores intencionales. En el análisis léxico se detecta el carácter @, se muestra su ubicación y se continúa con la lectura. Luego, en el análisis sintáctico, se encuentran tres errores por falta del símbolo » al final de una instrucción. El programa identifica dónde falta, lo agrega automáticamente mediante la recuperación a nivel de frase y continúa el análisis sin perder las demás instrucciones.



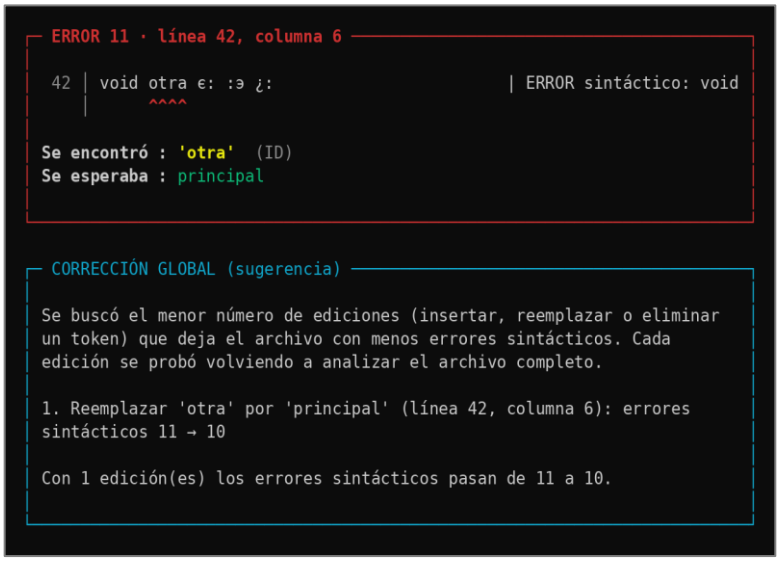
Prueba 7. Errores comunes con mensaje exacto

El mismo archivo contiene cuatro errores que se reconocen mediante la técnica de producción de error. Estos corresponden a una variable declarada sin tipo, un arreglo de tipo bool, una instrucción read sin paréntesis y un parámetro sin tipo. En cada caso, el programa muestra un mensaje indicando cuál es el problema y continúa con el análisis sin generar errores adicionales..

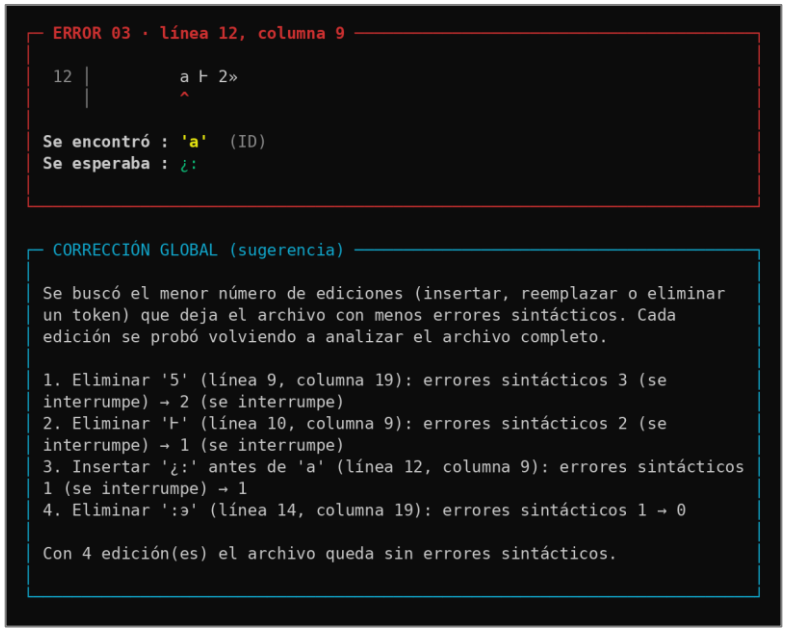


Prueba 8. Modo pánico y corrección global

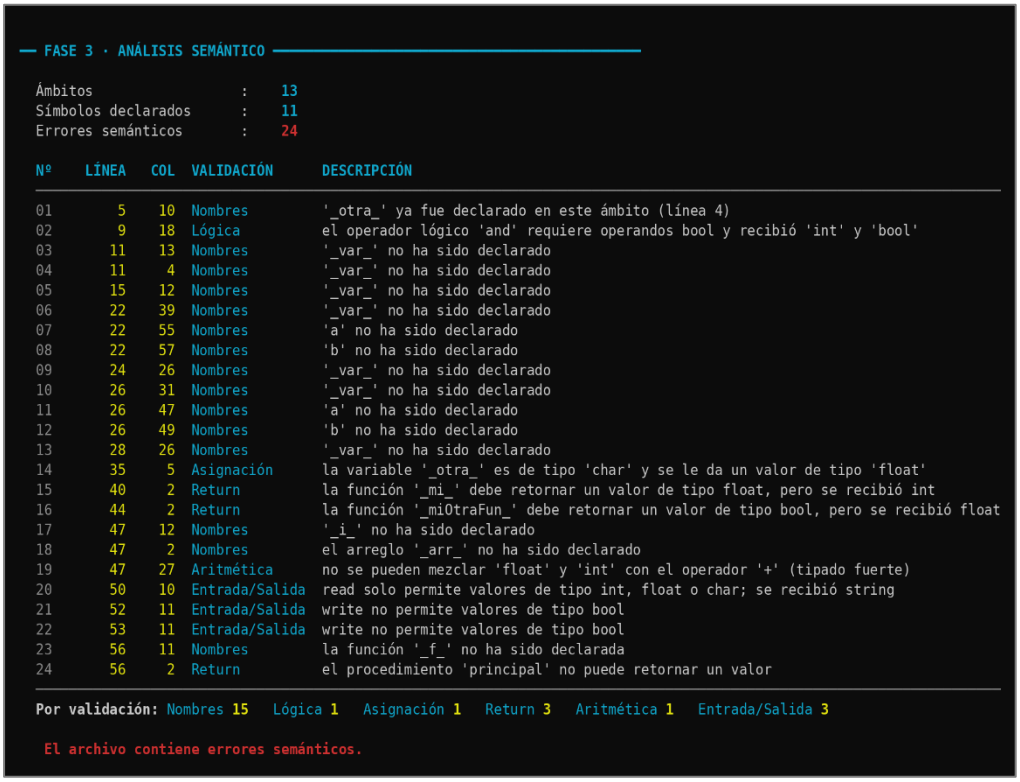
Cuando el programa no puede arreglar un error de forma exacta, usa el modo pánico: salta hacia adelante hasta un punto seguro y sigue. En la primera imagen la función void otra no es válida, porque void solo se permite en principal. Ese error no tiene técnica de recuperación específica y lo resuelve el modo pánico. Además, aparece un recuadro llamado Corrección global, que sugiere una forma de arreglar el archivo; en este caso propone cambiar el nombre otra por principal.



En la segunda imagen, otro archivo tiene varios errores juntos: un número de más, un símbolo de más, un bloque sin abrir y un paréntesis de más. La corrección global probó pequeños cambios (quitar, agregar o reemplazar una pieza), volvió a analizar todo el archivo después de cada cambio y encontró que con 4 ediciones el archivo queda sin errores sintácticos.



Prueba 9. Errores semánticos

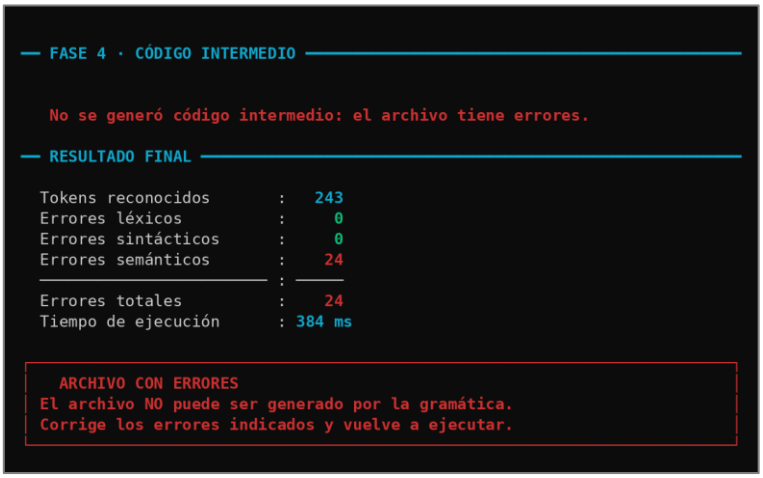


Se analiza el archivo ejemplo2.cmm, que es un ejemplo del profesor. La fase semántica encontró 24 errores, y cada uno aparece en una sola línea con su número, la línea y la columna del archivo, el tipo de revisión que falló y la explicación. Por ejemplo: un nombre repetido, una variable que no fue declarada, un operador

lógico con datos que no son bool, una variable de tipo char a la que se le da un valor decimal y una función que debe retornar un tipo y retorna otro. Al final hay un resumen con la cantidad de errores de cada tipo de revisión.

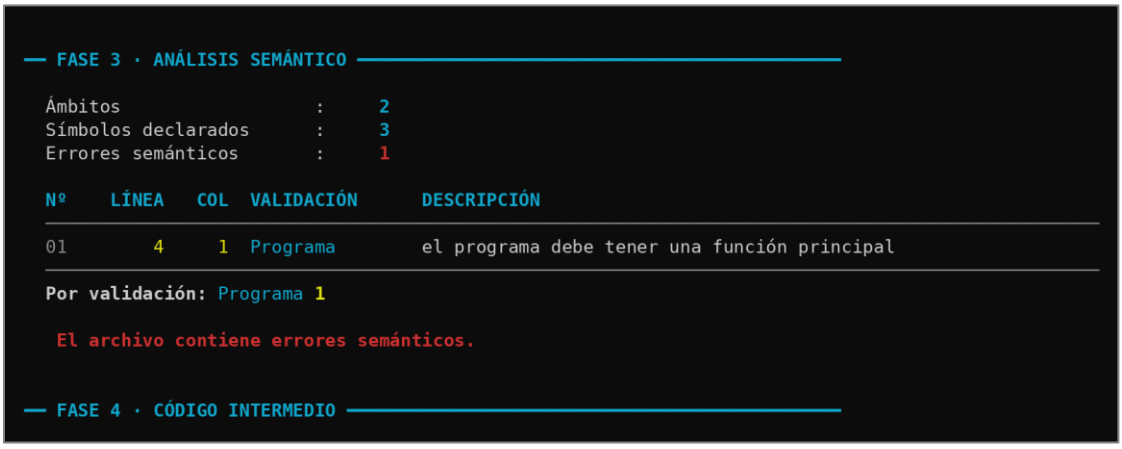
Prueba 10. Con errores no se genera código intermedio

En la misma ejecución, la fase 4 indica que no se generó código intermedio porque el archivo contiene errores. Al finalizar, el programa muestra el mensaje El archivo NO puede ser generado por la gramática, junto con un resumen de la cantidad de errores encontrados en cada fase.



Prueba 11. Programa sin la función principal

Se analiza el archivo prueba_sin_principal.cmm, que solo tiene una función llamada suma. El programa avisa con el error (el programa debe tener una función principal), porque todo programa necesita exactamente una función principal, y no genera código intermedio.



Descripción del problema

Hoy en día, muchos aparatos electrónicos, como los pequeños dispositivos que utilizan chips, necesitan instrucciones para funcionar. Para esto se utilizan lenguajes de programación. Como estos aparatos tienen poca memoria y capacidad de procesamiento, es importante contar con lenguajes sencillos y fáciles de procesar, que permitan escribir instrucciones sin utilizar todas las funciones de un lenguaje de programación común.

Por esta razón, en este curso se creó un lenguaje de programación propio. Este permite declarar variables, realizar operaciones, utilizar funciones, tomar decisiones con if, repetir instrucciones con while y for, trabajar con arreglos y leer y escribir información.

Para que la computadora pueda entender este lenguaje, se necesita un compilador. Un compilador es un programa que lee el código escrito y lo transforma en otras instrucciones que la computadora puede procesar. Este proceso se realiza en varias etapas.

Se puede comparar con revisar una carta. Primero, se leen e identifican las palabras; esto corresponde al análisis léxico. Después, se revisa que las oraciones estén bien construidas; esto es el análisis sintáctico. Luego, se comprueba que lo escrito tenga sentido; esto corresponde al análisis semántico. Por último, se transforma el programa en instrucciones más sencillas mediante la generación de código intermedio.

En el primer proyecto se desarrollaron las dos primeras etapas. En este segundo proyecto se conservaron y corrigieron esas etapas, y se agregaron el análisis semántico y la generación de código intermedio. Al finalizar, el programa debe indicar si el archivo puede ser generado por la gramática, es decir, si cumple con las reglas de escritura y tiene sentido.

El análisis semántico comprueba que las instrucciones tengan sentido, aunque estén escritas correctamente. Por ejemplo, la instrucción `val int a ⊢ 2.5`

tiene una estructura correcta, pero presenta un error porque intenta guardar un número decimal en una variable que solo acepta números enteros.

El lenguaje utiliza un tipado explícito y fuerte. Esto significa que cada variable debe declararse indicando su tipo y que no se pueden mezclar tipos de datos que no sean compatibles.

Entre las principales revisiones que realiza el programa están:

Comprobar que las variables y funciones estén declaradas antes de utilizarse y que no se repitan nombres dentro del mismo ámbito.

- Verificar que las operaciones utilicen los tipos de datos correctos.
- Revisar que las llamadas a funciones tengan la cantidad y los tipos de parámetros necesarios, y que los valores devueltos sean correctos.
- Comprobar que los arreglos tengan un tamaño válido y que sus posiciones existan.
- Verificar que las condiciones de `if`, `elif`, `while` y `for` sean de tipo `bool`.
- Comprobar que exista exactamente una función llamada `principal`.

Para guardar esta información, el programa utiliza una tabla de símbolos. Esta funciona como un registro donde se almacena información de cada variable y función, como su nombre, tipo, ubicación y tamaño. Además, se organiza por ámbitos, que representan los diferentes bloques del programa.

El manejo de errores. El programa debe identificar los errores léxicos, sintácticos y semánticos, indicar su línea y columna y continuar con el análisis cuando sea posible, en lugar de detenerse al encontrar el primero. Para esto, se utiliza la recuperación en modo pánico, que consiste en avanzar hasta encontrar un punto seguro para continuar. También se implementaron otras técnicas: recuperación a nivel de frase, producción de errores y corrección global.

La última etapa es la generación de código intermedio. Esta consiste en transformar el programa en instrucciones más sencillas, donde cada línea realiza una operación. Se utiliza el código de tres direcciones, en el que cada instrucción

contiene como máximo tres nombres, por ejemplo, t1 = a + b. El resultado se guarda en un archivo y mantiene la lógica del programa original. Si se encuentran errores durante el análisis, no se genera el código intermedio.

Diseño del programa

El programa se organizó en cuatro etapas: análisis léxico, sintáctico, semántico y generación de código intermedio. Todas se ejecutan en un mismo proceso y, al finalizar, se muestra un resumen de los resultados.

También se separaron las responsabilidades: Parser.cup revisa que las instrucciones sean correctas y Generador.java se encarga de escribir el código intermedio. Así, se evita generar código cuando existen errores y es más fácil realizar cambios.

Decisiones de diseño

- Las revisiones de sentido se hacen dentro de Parser.cup, mientras el programa se lee, y no en una etapa separada. Así se revisa cada parte justo cuando se reconoce.
- Cada error semántico lleva un tipo de revisión (Nombres, Aritmética, Arreglos…). Así cada error se puede relacionar con la documentación.
- Cada error semántico se muestra en una sola línea, con su línea y su columna, para que sea fácil de leer y de buscar en el archivo.
- Un error que ya se reportó no se repite. Si una variable ya tiene un error, las operaciones que la usan no producen más errores.
- La función principal puede ir en cualquier lugar del archivo, pero debe existir una sola.
- El código intermedio solo se genera cuando no hay errores.
- Los resultados se muestran con colores y recuadros para que cada fase se distinga fácilmente.

Algoritmos usados

Análisis léxico.

Funciona igual que en el primer proyecto. El analizador avanza por el archivo, carácter por carácter, y busca qué regla del lenguaje coincide con lo que va leyendo. Cuando una regla coincide, crea un token con su tipo, su texto, su línea y su columna. Los espacios y los comentarios se ignoran. Si aparece algo que no coincide con ninguna regla, se registra un error léxico y se sigue con el carácter siguiente.

Análisis sintáctico.

Java CUP convierte la gramática en una tabla de decisiones. El analizador lee los tokens de uno en uno y, con la tabla, decide qué regla del lenguaje se está cumpliendo.

Revisión de tipos.

Cada valor lleva consigo su tipo. Cuando el programa reconoce una operación, mira el tipo de los dos lados. Si son compatibles, crea un valor nuevo con el tipo del resultado. Si no, reporta el error y marca el resultado con el tipo especial error, para que las operaciones siguientes no repitan el aviso. También se calcula el valor de las constantes enteras por ejemplo 1 + 1 vale 2, lo que permite saber si la posición de un arreglo existe o si su tamaño es válido.

Tabla de símbolos con ámbitos.

Cada vez que aparece el símbolo ¿: se abre un ámbito nuevo, y al llegar a :? se cierra.

Los ámbitos forman un árbol: el global es el más grande y, dentro de él, están los de cada función y los de cada if, while o for. Para buscar un nombre, el programa empieza en el ámbito actual y, si no lo encuentra, sube al ámbito de afuera, hasta llegar al global. Si no aparece en ninguno, el nombre no fue declarado.

Al declarar uno nuevo, solo se revisa el ámbito actual, por lo que el mismo nombre se puede usar en bloques distintos. A cada variable se le asigna su tamaño en bytes y una dirección (global+0, pila+4…), sumando el tamaño de las variables anteriores.

Recuperación de errores.

El programa usa cuatro técnicas, de la más general a la más específica:

- **Modo pánico :** cuando hay un error, descarta tokens hasta llegar a un punto seguro, como el final de una instrucción, y continúa desde ahí.
- **Recuperación a nivel de frase:** cuando falta un símbolo pequeño (», :ɜ o :?), el programa lo agrega por su cuenta, avisa exactamente qué faltaba y sigue sin descartar nada. Solo se usa donde no hay duda de qué falta.
- **Producción de errores:** la gramática incluye reglas para errores comunes, como declarar una variable sin tipo. Así el mensaje es exacto y no se pierde el resto del análisis.
- **Corrección global:** cuando quedan errores que el modo pánico tuvo que resolver, el programa busca cuál es el menor número de cambios que dejan el archivo sin errores. Para cada error prueba quitar una pieza, agregar una pieza o cambiarla por otra, y después de cada prueba vuelve a analizar todo el archivo. Se queda con el cambio que elimina más errores y repite el proceso hasta seis veces. El resultado se muestra como una sugerencia: el archivo no se modifica.

Generación de código intermedio.

El código se escribe mientras el archivo se analiza, en el mismo orden en que el programa va terminando de reconocer cada parte. Estas son las ideas principales:

- **Temporales y lugar:** el resultado de cada operación se guarda en una variable temporal (t1, t2…). Cada valor recuerda dónde está su resultado, y así la operación siguiente sabe qué usar.
- **Etiquetas y saltos:** los if, while y for se convierten en etiquetas (L1, L2…) y saltos goto, ifFalse. Como un if puede tener otro dentro, el generador usa una lista donde guarda las etiquetas pendientes de cada uno, y siempre trabaja con la última que se agregó.
- **El incremento del for:** en el código fuente aparece antes del cuerpo, pero debe hacerse después de cada vuelta. El generador lo escribe cuando lo lee, lo recorta, lo guarda y lo vuelve a pegar al final del cuerpo.

- **Break:** salta a la etiqueta de salida del ciclo más cercano.
- **Funciones:** cada función empieza con una etiqueta con su nombre. Una llamada se escribe con un param por cada dato y un call que guarda el resultado. Antes de la primera función se escribe goto principal, para que el programa empiece en la función principal.
- **Arreglos:** un elemento se escribe como a[i]. Si se usa dentro de una operación, primero se copia a un temporal; si se usa como destino de una asignación, se deja como está.

Automatización.

El archivo build.bat busca las librerías, ejecuta Java CUP y JFlex, y compila todos los archivos de Java. El archivo run.bat pide el nombre de un archivo .cmm y ejecuta el programa. Con esto el usuario no tiene que escribir cada comando, lo que hace el proceso más sencillo y con menos posibilidad de equivocarse.

Librerías usadas

El proyecto utiliza dos herramientas externas: JFlex 1.9.1, para generar el analizador léxico, y Java CUP 11b, para generar el analizador sintáctico y realizar las revisiones semánticas. Ambas vienen incluidas en el repositorio.

También se utilizan librerías propias de Java para:

- Manejo de archivos: leer el código fuente y guardar los resultados.
- Manejo de texto: mostrar mensajes, errores y generar reportes.
- Colecciones: almacenar tokens, errores, variables y funciones.
- Validaciones: buscar patrones y revisar los mensajes de error.
- Fechas y horas: registrar cuándo se generan los reportes.
- Entrada de datos: recibir las opciones que selecciona el usuario.
- Java CUP: manejar los tokens y la comunicación entre los analizadores.
- Java JDK 21.0.12 para compilar y ejecutar todo el proyecto

Análisis de resultados

Objetivos alcanzados

- Se completaron las cuatro fases del compilador: análisis léxico, sintáctico, semántico y generación de código intermedio.
- Se corrigieron y mejoraron los análisis léxico y sintáctico del primer proyecto.
- Se implementó el análisis semántico con tipado explícito y fuerte.
- Se construyó una tabla de símbolos con ámbitos anidados.
- Se comprobaron los resultados con los dos ejemplos del profesor, identificando los 35 y 24 errores semánticos correspondientes.
- Se implementó el manejo y la recuperación de errores léxicos, sintácticos y semánticos.
- Se incorporaron las técnicas de recuperación en modo pánico, a nivel de frase, producción de errores y corrección global.
- Se agregó un resumen final con el resultado de cada fase y la cantidad de errores encontrados.
- Se implementó la generación de código intermedio de tres direcciones.
- Se automatizaron la compilación y la ejecución mediante los archivos build.bat y run.bat.
- Se realizaron pruebas para comprobar el funcionamiento de cada parte y evitar errores después de los cambios.

Objetivos no alcanzados

- Manejo de clases

El enunciado da puntos extra por incluir clases en el lenguaje, pero pide que funcionen en las cuatro fases. No se implementó.

- Recuperación a nivel de frase en todos los lugares.

Esta técnica agrega por su cuenta un símbolo que falta, pero solo se pudo aplicar en algunos lugares: al final de las instrucciones, antes de abrir un bloque y

al final de una función. No se aplicó a los paréntesis dentro de una operación, a los corchetes de un arreglo ni a los bloques que están dentro de otros.

Razones por las que no se alcanzaron

No se implementaron las clases porque el enunciado no explica cómo deben declararse ni utilizarse. Además, habría sido necesario modificar varias partes del compilador. Se decidió dar prioridad a completar los requisitos obligatorios.

La recuperación a nivel de frase se limitó porque, al faltar símbolos dentro de una operación, Java CUP generaba conflictos y no podía crear el analizador. Por eso, en esos casos se utiliza el modo pánico.

Justificación de toma de decisiones

Por qué la revisión semántica se hizo dentro de Parser.cup

El análisis semántico se implementó en Parser.cup, donde cada revisión tiene su propio método y se ejecuta desde la regla correspondiente. TablaSimbolos.java se encarga de guardar y buscar información, mientras que Parser.cup detecta los errores.

Cómo se guarda la información de cada valor

Cada valor se representa con un objeto de la clase Atrib, que guarda su tipo, el lugar donde se encuentra y su valor cuando es una constante entera. Estos datos se utilizan para las revisiones y la generación de código intermedio.

Cómo se hace la validación de tipos

Cada operación llama a un método que comprueba si los tipos de datos son correctos. Primero revisa si ya existe un error y luego verifica si la operación está permitida. Se aplican las siguientes reglas:

- **Aritmética:** los dos lados deben ser int o float, y no se pueden mezclar. La división entera (//) solo trabaja con int, la división decimal (/) solo con float y la potencia solo acepta un exponente positivo.

- **Comparaciones:** para <, >, <= y >= los dos lados deben ser int o los dos float. Para == y != se aceptan int, float, bool y char. Los textos (string) no se comparan.
- **Operaciones lógicas:** λ, θ y Σ solo aceptan valores bool.
- **Asignaciones y declaraciones:** el valor debe tener el mismo tipo que la variable.
- **Arreglos:** el tamaño debe ser un entero mayor que cero, los valores iniciales deben ser del tipo del arreglo, no puede haber más valores que el tamaño y la posición debe ser un entero que exista, empezando en 0.
- **Funciones:** una llamada debe tener la cantidad y el tipo de datos que la función pide. Una función no puede devolver string, sus parámetros no se pueden repetir, un return debe devolver el tipo de la función y toda función con tipo debe tener al menos un return. La función principal es void y no puede devolver valores.
- **Condiciones y ciclos:** la condición de if, elif, while y for debe ser bool, y break solo se permite dentro de un ciclo.
- **Entrada y salida:** read acepta int, float o char, y write acepta int, float o string.

Por qué se usó el tipo error

Se utilizó el tipo error para marcar los valores que ya tienen algún problema. Así, las siguientes operaciones reconocen ese error y evitan mostrar avisos repetidos o falsos. Esto facilita identificar el error original sin generar confusiones.

La tabla de símbolos y los ámbitos

Cada bloque tiene su propio ámbito, donde se guardan las variables y funciones declaradas. Esto permite repetir nombres en bloques diferentes. La búsqueda comienza en el ámbito actual y continúa hacia los superiores.

Cada símbolo guarda su nombre, tipo, categoría, tamaño, desplazamiento en memoria y ubicación donde fue declarado. Los ámbitos se organizan como un árbol. Además, las funciones se registran antes de analizar su contenido para permitir que puedan llamarse a sí mismas.

Decisiones sobre las reglas del lenguaje

- **write:** acepta int, float y string.
- **Condiciones:** los if deben utilizar valores bool.
- **Comparaciones:** los corchetes son opcionales.
- **Función principal:** puede aparecer en cualquier lugar, pero debe existir una sola.
- **print:** se eliminó porque no forma parte de las instrucciones solicitadas; se utiliza write.

Decisiones sobre la recuperación de errores

Se utilizó el modo pánico por ser obligatorio. También se agregaron la recuperación a nivel de frase y la producción de errores para casos específicos. La corrección global sugiere cambios, pero no modifica el archivo automáticamente.

Decisiones sobre el código intermedio

- El código intermedio se genera en Generador.java para mantenerlo separado de la gramática.
- El código solo se genera cuando no existen errores.
- Cada instrucción utiliza como máximo tres nombres, como lo solicita el enunciado.
- Las operaciones largas se dividen en pasos utilizando variables temporales.
- Los operadores λ y θ siempre evalúan sus dos lados.
- Las etiquetas se organizan para permitir condiciones y ciclos dentro de otros bloques.
- No se realizan optimizaciones para facilitar la relación entre el código generado y el programa original.

Bitácora

La bitácora se genera automáticamente con Git.

https://github.com/Alice-Arias/PY2-Analisis-Semantico-y-Generacion-Codigo-Intermedio-Compiladores/commits